

Progetto NCIS – SDN DoS Detection e Mitigation

Studenti: Davide Cucciniello DE000034, Raffaele Di Santo

Link repository Github: <https://github.com/Mhanz-0/ncis.git>

1 OBIETTIVI

Il progetto consiste nel simulare diversi tipi di attacchi, tra cui DoS e DDoS, su un ambiente emulato con Mininet, Open vSwitch, Ryu e OpenFlow 1.3.

Gli obiettivi principali del progetto erano:

- limitare l'overblocking del traffico in seguito a rilevazione di flussi malevoli
- implementare soglie non statiche e quindi adattabili al comportamento della rete
- rendere più modulari le funzionalità del controller SDN
- simulare più scenari di attacco
- rendere più flessibili le politiche di blocco/sblocco flussi

2 ARCHITETTURA DEL PROGETTO

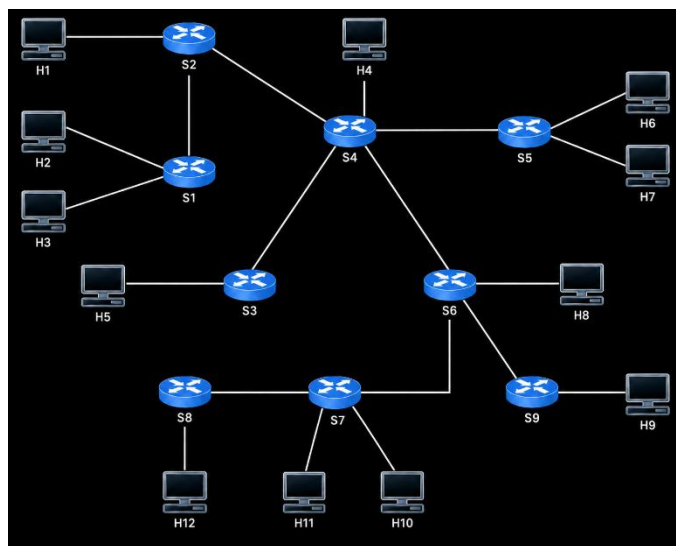
Il progetto è suddiviso principalmente in 3 parti: nella cartella **main** troviamo gli script che definiscono la topologia della rete e i vari scenari di attacco implementati; nella cartella **controller** troviamo i due controller SDN usati per il progetto; nella cartella **components** troviamo le funzionalità a disposizione del controller SDN. Altre cartelle includono i file di log e l'interfaccia Flask. Infine è presente un file di shell per avviare il programma da terminale denominato "*start_project.sh*".

2.1 TOPOLOGIA

Per la topologia di rete abbiamo scelto di utilizzare 12 **host** e 9 **switch**, dopodiché abbiamo effettuato tutti i collegamenti facendo attenzione a non creare cicli all'interno della rete. Ogni elemento è stato aggiunto manualmente tramite le funzioni di Mininet "*addHost*", "*addSwitch*" e "*addLink*".

Link host-switch	5Mbit/s, 0.5ms delay
Link switch-switch	25Mbit/s, 2.5ms delay

Alleghiamo di fianco un'immagine rappresentativa della topologia, dove possiamo constatare l'assenza di cicli interni alla rete e l'impiego degli switch S4 ed S5 come colli di bottiglia della rete.



2.2 PROFILI DI ATTACCO

Abbiamo simulato 3 diversi tipi di attacco sulla rete, la cui esecuzione è rimandata alle funzioni definite nello script “*scenarios.py*”: **stealth_attack()**, **burst_attack()** e **custom_attack()**.

Ogni scenario prevede la presenza di host benevoli che trasmettono traffico legittimo, al fine di popolare la rete non soltanto con gli attacchi.

2.2.1 Stealth attack

Simula un attacco “stealth” ossia progressivo: l’host h1 è la vittima dell’attacco e ogni 3-4 secondi viene scelto un host random (tra quelli che non generano traffico benevolo, in questo caso sono esclusi h1, h3, h5 ed h10) il quale genera traffico malevolo tramite la funzione *hping3*. Più nello specifico, esso invierà ogni millisecondo pacchetti da 800 byte, la durata totale dell’attacco è di 95 secondi e il protocollo di trasporto scelto è sorteggiato tra TCP e UDP.

Due variabili importanti che possono essere regolate in uno *stealth_attack* sono la durata complessiva dell’attacco (*time_stealth*) e il numero di host che partecipano all’attacco (*max_attack_stealth*), che sarebbe opportuno tenere limitato almeno a ¼ della durata complessiva poiché ogni inserimento progressivo dura come visto prima 3-4 secondi.

2.2.2 Burst attack

Simula un attacco improvviso: l’host h1 attacca l’host h6, mentre altri host (in questo caso h2, h3, h4 ed h5) generano traffico legittimo. L’attacco dura 20 secondi e consiste in pacchetti da 3000 byte UDP inviati in flood verso h6.

La variabile *max_attack_burst*, settata di default a 10, indica il numero di “burst”, ossia di attacchi, che deve generare h1. È opportuno quindi che non superi i 15 attacchi, poiché l’attesa tra un attacco e l’altro è settata al più a 8 secondi e il server di h6 è aperto per una durata di 120 secondi.

2.2.3 Custom attack

Simula un attacco DDoS condotto da 4 attaccanti diversi all’interno della rete (h2, h5, h8 e h9) ai danni dell’host h6. 2 attacchi sono TCP e 2 sono UDP, i pacchetti di grandezza 1400/1450 byte sono inviati ogni 250 microsecondi, per una durata totale di 95 secondi di attacco. I singoli attacchi sono lanciati come thread per poter eseguire contemporaneamente.

2.3 CONTROLLER

Abbiamo fatto uso di 2 controller SDN diversi: un primo controller di base che gestisce il forwarding tra gli host installando le flow entry a livello L2 ed un secondo controller che si occupa del monitoring della rete e dell’installazione di regole di mitigation in seguito a rilevazione di attacchi.

2.3.1 simple_switch13.py

È un controller SDN scritto con Ryu che fa uso di OpenFlow 1.3 a cui è delegata la parte di forwarding della SDN. Implementa un servizio learning switch per gli host della rete: quando uno switch si connette al controller è installata una *table-miss rule* (priorità 0), per cui lo switch dovrà inoltrare al controller ogni pacchetto che non corrisponde a nessuna flow entry. Il controller risponde installando una nuova flow entry apposita (priorità 1).

2.3.2 controller.py

È un controller SDN che gestisce diversi tipi di funzionalità racchiuse nella cartella *components*. Invia periodicamente richieste di statistiche a tutti gli switch connessi della rete ogni *MONITOR_INTERVAL*, settato di default ad 1 secondo. Osserviamo la pipeline dell'handler del controller in seguito alla ricezione di statistiche da parte degli switch:

1. filtraggio dei soli flussi a priorità 1, ossia quelli generati dal primo controller
2. calcolo del *packet_diff* (ossia il numero di pacchetti nuovi passati su un determinato flusso dall'ultima richiesta di statistiche a quella attuale) e del *flow_rate* che corrisponde al throughput di quel flusso nell'intervallo di tempo tra due richieste di statistiche.
3. dopo che tutti gli switch hanno risposto, si calcola la soglia adattiva
4. si fa *detection* usando la soglia calcolata al punto precedente e che cambia dinamicamente con l'evoluzione della rete
5. se necessario si richiama la *mitigation* sui flussi considerati sospetti

3 FUNZIONALITÀ DEL CONTROLLER

MODULO	FUNZIONALITÀ
detection.py	calcola la soglia adattiva in una delle 2 modalità possibili ("percentile" e "average&std_dev", da impostare al lancio del comando di start) e classifica i flussi come sospetti o meno confrontandoli con la soglia e con un <i>MIN_RATE</i> basso prestabilito
mitigation.py	mantiene contatori di sospetto per ogni flusso e dopo 3 rilevazioni consecutive attiva lo stato d'allarme per quel determinato flusso; installa una drop rule OpenFlow (priorità 10); calcola il tempo di blocco con backoff esponenziale per flussi recidivi
monitoring.py	registra e rimuove gli switch connessi al controller e invia richieste di statistiche OpenFlow agli switch connessi, di tipo <i>OFPFLOWStatsRequest</i>
policy.py	legge da una lista di flussi <i>whitelistati</i> o <i>blacklistati</i> e permette al controller di sapere se un flusso appartiene ad una delle due liste; permette l'aggiunta e la rimozione di flussi tramite interfaccia Flask
statistics.py	estrae la terna (in_port, eth_src, eth_dst) per identificare un flusso e calcola diversi tipi di statistiche necessarie al controller; salva le statistiche in JSON e stampa un plot che mostra l'evoluzione nel tempo del <i>flow_rate</i> medio, massimo e del valore di soglia

3.1 Altre funzionalità

È stata aggiunta un'interfaccia web basata su Flask, avviata separatamente rispetto alla rete e al controller SDN, al fine di avere uno strato applicativo che semplifica la visualizzazione di log e statistiche in tempo reale e permette di modificare *whitelist* e *blacklist* durante l'esecuzione.

L'interfaccia assieme al template HTML è contenuta nella cartella *flaskInterface*.

4 AVVIO DEL PROGETTO

Il programma può essere comodamente lanciato da terminale dopo aver installato le dipendenze python numpy, pandas, matplotlib e flask.

Una volta all'interno della cartella del progetto, è importante copiare i 5 script di funzionalità in una cartella denominata "components" e l'interfaccia flask in un'altra denominata "flaskInterface".

Il lancio del programma avviene tramite l'esecuzione su terminale dello shell di start seguito dalla modalità di calcolo della soglia che si intende testare, dopo averlo reso eseguibile col seguente comando:

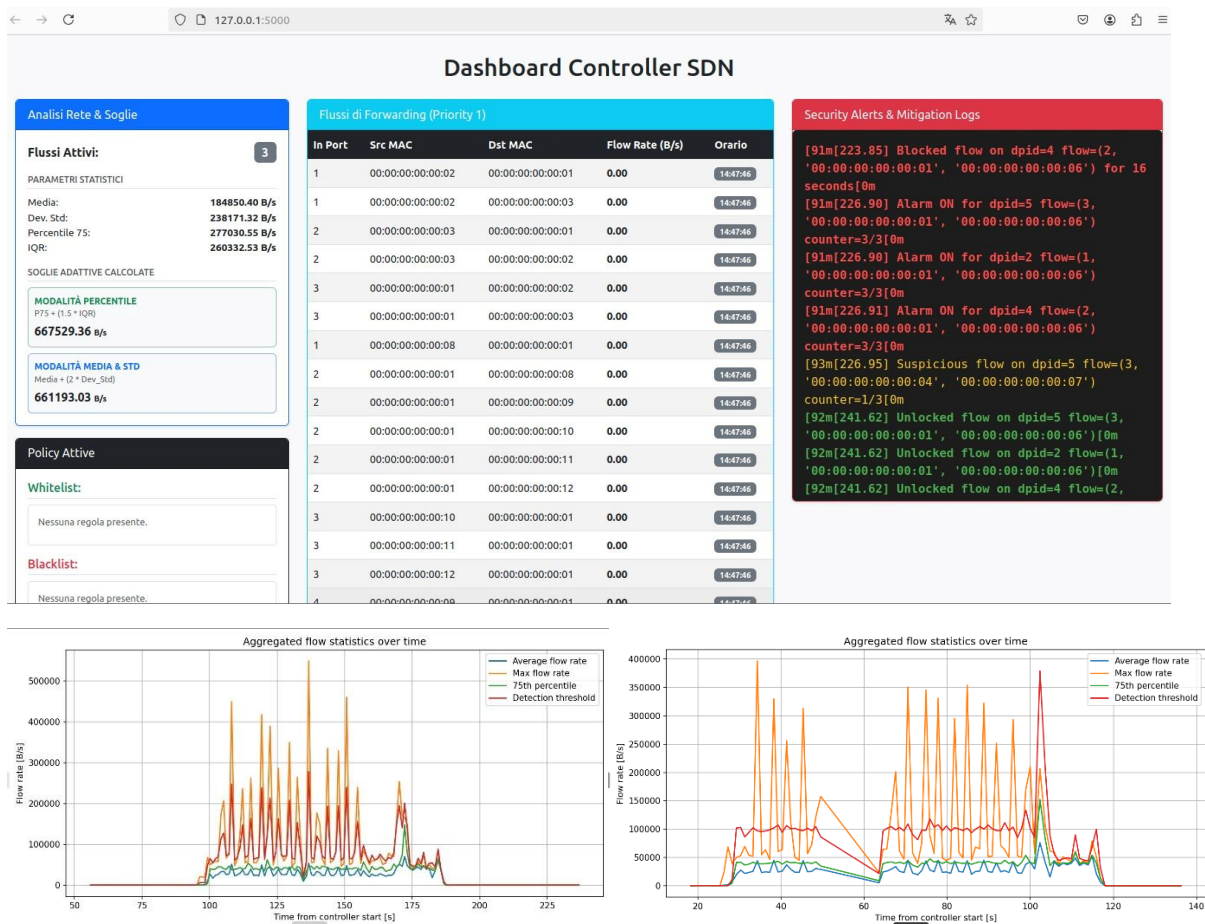
```
chmod +x start_project.sh
```

All'interno della CLI Mininet è possibile lanciare i seguenti comandi:

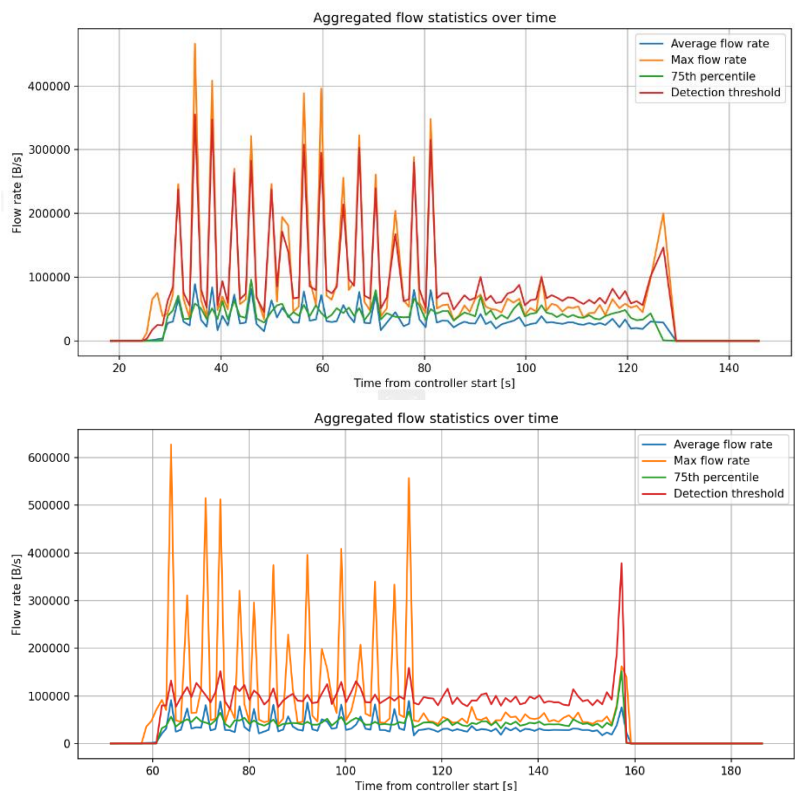
- `py scenarios.run("stealth")`
- `py scenarios.run("burst")`
- `py scenarios.run("custom")`
- `py scenarios.run("all")`

5 RISULTATI

5.1 Simulazione burst_attack



5.2 Simulazione custom_attack



5.3 Simulazione blacklist

Dopo aver fatto installare al `simple_switch13` il flow di forwarding tra `h1` e `h4` per individuare la porta dello switch da bloccare, abbiamo aggiornato la *blacklist* come si può vedere in foto e riprovato a lanciare traffico tra i due host, col risultato che il flusso è stato bloccato.

```
mininet> h1 ping -c 3 h4
PING 10.0.0.4 (10.0.0.4) 56(84) bytes of data.
--- 10.0.0.4 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2082ms
```

Flussi Attivi: 0

PARAMETRI STATISTICI

Media: 0.00 B/s
Dev. Std: 0.00 B/s
Percentile 75: 0.00 B/s
IQR: 0.00 B/s

SOGGIE ADATTIVE CALCOlate

MODALITÀ PERCENTILE
 $PFS = (1.5 * IQR)$
0.00 B/s

MODALITÀ MEDIA & STD
 $Media = (2 * Dev_Std)$
0.00 B/s

Policy Attive

Whitelist:

Nessuna regola presente.

Blacklist:

Porta: 1
Da: 00:00:00:00:00:01
A: 00:00:00:00:00:04
Motivo: Nessuna motivazione
Attivo

In Port	Src MAC	Dst MAC	Flow Rate (B/s)	Orario
1	00:00:00:00:00:01	00:00:00:00:00:04	0.00	16:15:39
3	00:00:00:00:00:04	00:00:00:00:00:01	0.00	16:15:39
1	00:00:00:00:00:04	00:00:00:00:00:01	0.00	16:15:39
2	00:00:00:00:00:01	00:00:00:00:00:04	0.00	16:15:39

```
[92m[266.89] Unlocked flow on dpid=2 flow=(1, '00:00:00:00:00:01', '00:00:00:00:00:04')[0m
[91m[268.15] Blocked flow on dpid=2 flow=(1, '00:00:00:00:00:01', '00:00:00:00:00:04') for 30 seconds[0m
[92m[298.16] Unlocked flow on dpid=2 flow=(1, '00:00:00:00:00:01', '00:00:00:00:00:04')[0m
[91m[298.85] Blocked flow on dpid=2 flow=(1, '00:00:00:00:00:01', '00:00:00:00:00:04') for 30 seconds[0m
[92m[329.03] Unlocked flow on dpid=2 flow=(1, '00:00:00:00:00:01', '00:00:00:00:00:04')[0m
[91m[330.14] Blocked flow on dpid=2 flow=(1, '00:00:00:00:00:01', '00:00:00:00:00:04') for 30 seconds[0m
[92m[361.96] Unlocked flow on dpid=2 flow=(1, '00:00:00:00:00:01', '00:00:00:00:00:04')[0m
[91m[362.32] Blocked flow on dpid=2 flow=(1, '00:00:00:00:00:01', '00:00:00:00:00:04') for 30 seconds[0m
```